

Research Data Logging System

Game-Based Learning Study — Game-LLM-Interactive Condition

Valentino Clerx

Fontys Master of Applied IT

2026

1. Introduction

The research study involves multiple experimental conditions in which participants interact with a game-based learning environment. To support quantitative analysis of player behaviour, NPC interaction quality, and narrative progression, the game must capture raw event data throughout each session. This document details the design of the three-component Data Logging System that fulfills this requirement.

The logging system captures three distinct categories of data: raw player input interactions, scripted dialogue progression, and full AI conversation histories. Together, these data streams are intended to enable post-hoc analysis of learning behaviour, engagement patterns, and the effectiveness of the LLM-powered NPC across participant sessions.

All three loggers share a common architectural pattern and are implemented as independent Godot Node scripts operating in parallel throughout a session.

2. System Architecture

The logging system follows a Dual-Write pattern: every log entry is written to both a local CSV file on the participant's machine and forwarded to a cloud-hosted Google Sheet via a Google Apps Script web endpoint. This design ensures that data is never lost due to a network failure during a session, while still centralising raw data for analysis without manual file retrieval.

The three loggers operate independently of one another, each targeting a separate Google Sheet and writing to a separate local CSV file. They communicate with shared singleton managers (GameVersionManager, GameFlowManager, DebugConfig) to enrich log entries with session-level context such as game version, loop count, and scene name.

2.1 Component Breakdown

- **PlayerDataLogger (Input Logger):** PlayerDataLogger (Input Logger):

Captures all raw mouse and keyboard interactions. Detects clicks, drag start/end events, and Enter key presses. Uses Godot's physics query system to identify the game object under the cursor at the time of interaction.

- **hc_Dialogue_Logger (Dialogue Logger):** hc_Dialogue_Logger (Dialogue Logger):

Records the progression of the scripted HC (Health Check) dialogue tree. Each time the player advances to a new dialogue step, the step's path identifier is logged. This allows reconstruction of which narrative branches were visited.

- **AiLogger (AI Conversation Logger):** AiLogger (AI Conversation Logger):
- Captures the complete conversation history between the player and the LLM-powered NPC Veterinarian at the moment a conversation concludes. Also records which API host (Portkey or Google Direct) served the session, linking directly to the Host Failsafe System.

2.2 Shared Identity System

All three loggers implement an identical `_get_or_create_user_id()` function. On first launch, a pseudonymous identifier is generated in the format `USR-[10 alphanumeric characters]` and written to a shared file at `user://user_id.txt`. On subsequent launches, this file is read to restore the same identifier. Because all three loggers read from the same file path, every row across all three data streams for a given participant carries the same UserID, enabling cross-logger join operations during analysis.

3. Design

3.1 Log Entry Schema

Each logger produces rows conforming to a shared base schema, with logger-specific columns appended. The base columns present in all three loggers are described below.

Column	Source	Present In
Timestamp	Time.get datetime string from system()	All three loggers
UserID	user://user_id.txt (shared)	All three loggers
GameVersion	GameVersionManager.get_version()	All three loggers
Loop	GameFlowManager.get_loop_count() + 1	All three loggers
Scene	get_tree().current_scene.name	All three loggers
Interaction	Detected input event type	PlayerDataLogger only
Object	Physics/UI query result	PlayerDataLogger only
DialogueStep	Dialogue node path string	hc Dialogue Logger only
LLM Host	GeminiManager.current_host	AiLogger only
Full Conversation	Complete chat history string	AiLogger only

3.2 The Dual Write Pipeline

When a loggable event occurs, the relevant public logging function is called (e.g., `log_interaction()`, `log_dialogue_step()`, or `log_ai_conversation()`). The function first checks `DebugConfig.logging_enabled` to allow logging to be globally suppressed during development. If enabled, the entry is constructed, then two operations are performed in sequence:

- **Local Write:** `_write_to_file()` appends the CSV row to the local log file immediately and synchronously, ensuring data persistence regardless of network state.
- **Cloud Queue:** The entry is appended to `request_queue` and `_process_queue()` is called to attempt asynchronous cloud synchronisation.

3.3 Asynchronous Queue & HTTP Management

Cloud synchronisation is handled by a serial request queue to prevent concurrent HTTP calls from conflicting. The `is_queue_processing` flag acts as a mutex: only one HTTP request may be in-flight at any time. Upon completion, `_on_request_completed()` releases the flag and calls `_process_queue()` again, draining the queue entry-by-entry until it is empty.

The GDScript pattern used is:

```
func _process_queue() -> void:
    if is_queue_processing or request_queue.is_empty(): return
    is_queue_processing = true
    var row_data = request_queue.pop_front()
    http_request.request(GOOGLE_SCRIPT_URL, headers, HTTPClient.METHOD_POST, body)
```

In the event of an HTTP error, the queue continues to the next entry rather than halting. This prioritises throughput — a single failed sync does not block subsequent events from being uploaded. The local CSV backup ensures that the failed entry is not lost.

The `PlayerDataLogger` additionally implements a 1-second retry delay using Godot's timer coroutine (`await get_tree().create_timer(1.0).timeout`) when the `request()` call itself fails at the OS level, providing a basic backoff before retrying.

3.4 Player Input Detection (`PlayerDataLogger`)

The `PlayerDataLogger` overrides the `_input()` callback to intercept all engine-level input events. Three interaction types are captured:

- **Mouse Click:** Logged when the left mouse button is released without having exceeded the drag threshold distance (10.0 pixels).
- **Mouse Drag (Start):** Logged when the mouse moves more than 10.0 pixels while the left button is held, transitioning the state to `is_dragging = true`.
- **Mouse Drag (End):** Logged on left button release when `is_dragging` is true, recording both the origin object and the object under the cursor at release time.
- **Key Press (Enter):** Logged when either the main or numpad Enter key is pressed, capturing text submission events.

Object identification is handled by `_find_clicked_object()`, which performs a two-stage lookup: first checking Godot's UI layer for a hovered `Control` node, then falling back to a 2D physics point query at the global mouse position. The physics query handles the `InteractionAreas` naming convention used in the game's scene structure, traversing up the node hierarchy to return a meaningful parent object name.

3.5 AI Log CSV Escaping (`AiLogger`)

The full conversation string logged by `AiLogger` may contain commas and quotation marks, which would corrupt a naive CSV row. The logger applies standard RFC 4180 CSV escaping: all double-quote characters within the conversation string are doubled (`""`), and the entire field is wrapped in double quotes before being written. This ensures the conversation field is parsed as a single cell when the CSV is imported into analysis tools.

4. Implementation Details

4.1 HTTP Node Instantiation

The three loggers differ slightly in how they acquire their HTTPRequest node. The PlayerDataLogger uses `add_child(HTTPRequest.new())` to self-instantiate its node at runtime, making it fully self-contained and requiring no scene setup. The `hc_Dialogue_Logger` and `AiLogger` instead rely on an `@onready` var referencing an HTTPRequest node that must be present as a child in the scene tree. The `AiLogger` additionally sets a longer timeout (30.0 seconds versus 15.0 seconds) to accommodate the larger payload size of full conversation histories..

4.2 Local File Handling

All three loggers use an identical file-open strategy for `_write_to_file()`. The file is first opened in `READ_WRITE` mode to seek to the end and append. If this fails (i.e., the file does not yet exist), it falls back to `WRITE` mode to create it. The CSV header row is written once on first launch, guarded by a `FileAccess.file_exists()` check. This approach does not use Godot's `FileAccess.WRITE_READ` constant intentionally, as that mode would truncate the file on open.

4.3 Session Context Enrichment

Each log entry is enriched at write time with session-level context sourced from three shared singletons. `GameVersionManager.get_version()` ensures that data collected across different builds of the game can be filtered or stratified. `GameFlowManager.get_loop_count()` provides the current game loop iteration, allowing researchers to compare first-loop versus repeat-loop behaviour within a single session. The current scene name provides spatial context, indicating which part of the game the event occurred in.

5. Design Considerations

5.1 Data Volume & Logger Granularity

The PlayerDataLogger captures every mouse click, drag, and Enter keypress during a session. In a typical playtesting session, this may generate a high volume of rows. The queue-based upload system is designed to handle this gracefully, serialising requests so that bursts of interaction do not result in concurrent HTTP calls. For the purposes of this research study, completeness of the interaction record is prioritised over data economy.

5.2 Data Linkage Across Loggers

Because all three loggers share the same `UserID`, `GameVersion`, `Loop`, and `Scene` columns, rows from separate sheets can be temporally aligned and joined during analysis. For example, a researcher can isolate all interaction events that occurred during the same scene and loop iteration in which an AI conversation was logged, enabling precise behavioural analysis around NPC interactions.

5.3 Logging Toggle & Research Integrity

All three loggers gate every write operation behind a `DebugConfig.logging_enabled` check. This allows the development team to disable logging entirely during testing or debugging without modifying any logger code. Crucially, this toggle ensures that internal testing sessions do not contaminate the research dataset. For live playtesting sessions with study participants, this flag must be set to true prior to distribution.

5.4 Security Considerations

Google Apps Script endpoint URLs are currently stored as plaintext constants within each logger script. In the context of a controlled research study with a closed participant group, this is considered acceptable. However, exposure of these URLs could allow third parties to submit arbitrary data to the research sheets. For a wider deployment or publication of the game, these endpoints should be moved to an encrypted configuration file or obfuscated build variable, consistent with the recommendation made in the Host Failsafe System document regarding API key storage.